

EduGenie: Google Gemini Powered Learning Assistant

Submitted by: Arumparithi B | Team size: 1

Project Description:

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to ask questions and receive smart, concise answers; understand complex concepts through simplified explanations; generate quizzes from topics or text; receive personalized learning recommendations; and summarize large educational passages.

Built with FastAPI for the backend and a simple HTML + CSS + JavaScript frontend, EduGenie uses Google Gemini through the Google Gen AI SDK for cloud intelligence and the lightweight LaMini-Flan-T5-783M model for optional local concept explanations, so it runs well even on modest hardware such as a Mac M1 or an ordinary laptop.

Scenarios

Scenario 1: A student who wants to know about oceans and rivers asks EduGenie "Which is the largest ocean?" and instantly receives a concise answer.

Scenario 2: A student wants to check their understanding of "The Pythagoras Theorem", selects Generate Quiz and answers three MCQs with instant feedback and a score.

Scenario 3: A learner exploring SQL requests a learning path and receives a structured plan from beginner to advanced topics with timelines, resources and practice projects.

Technical Architecture:

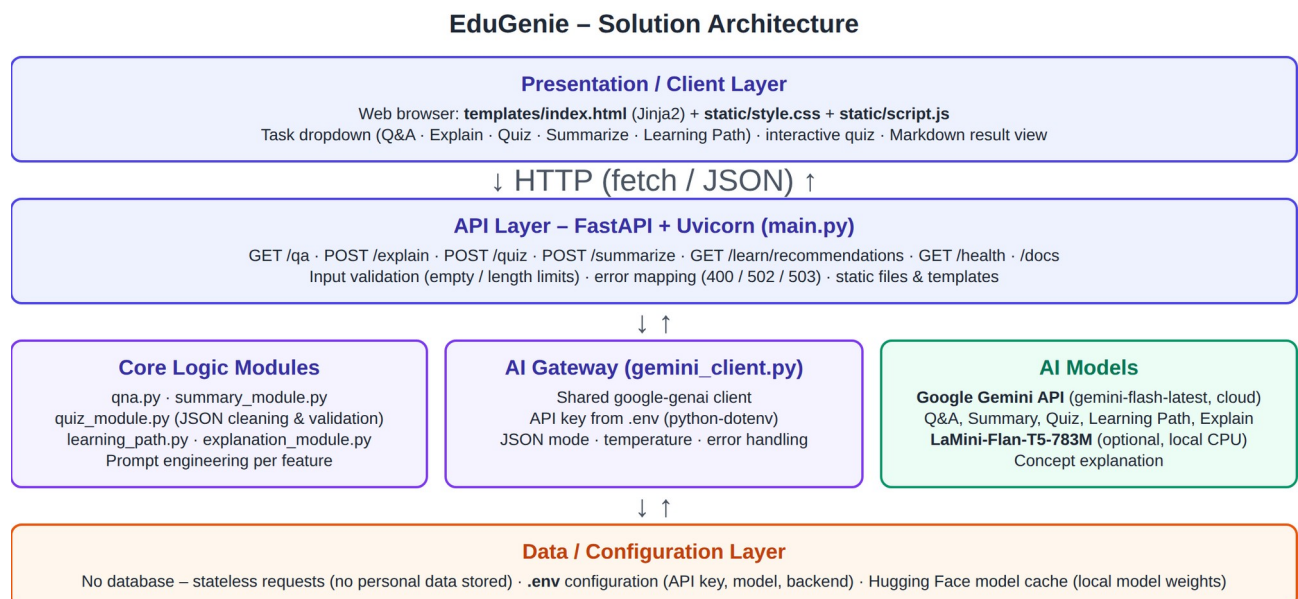


Figure 1: EduGenie technical architecture

Description: EduGenie uses a modular three-tier architecture. The browser-based frontend sends JSON requests to a FastAPI backend, which validates input, builds a task-specific prompt and calls Google

Gemini through a shared client (or the local LaMini-Flan-T5 model for explanations). Responses are validated and returned as JSON for display. No database is used - EduGenie is stateless and stores no personal data.

Pre-requisites:

- **Python 3.10+:** <https://www.python.org/downloads/> (tick "Add Python to PATH")
- **FastAPI Framework:** <https://fastapi.tiangolo.com/>
- **Google Gemini API key:** <https://aistudio.google.com/app/apikey> - Google AI for Developers documentation: <https://ai.google.dev/>
- **Uvicorn (ASGI server):** pip install uvicorn
- **Jinja2 (HTML templating):** pip install jinja2
- **HTML, CSS and JavaScript:** basic templating in /templates and /static
- **Git & VS Code:** version control and development environment

Project Workflow:**Milestone 1: Model Selection and Architecture**

- Activity 1.1: Generate a Gemini API key and configure it securely in .env.
- Activity 1.2: Select AI models - Gemini (cloud) and LaMini-Flan-T5-783M (local).
- Activity 1.3: Define the architecture and folder structure.
- Activity 1.4: Set up the development environment.

Milestone 2: Core Functionalities Development

- Activity 2.1: Implement the five feature modules.
- Activity 2.2: Build the FastAPI backend endpoints.

Milestone 3: Frontend Development

- Activity 3.1: Build the web interface (HTML, CSS).
- Activity 3.2: Live integration with JavaScript.

Milestone 4: Deployment and Testing

- Activity 4.1: Run locally with Uvicorn.
- Activity 4.2: Functional, automated and performance testing.

Milestone 5: Conclusion

Milestone 1: Model Selection and Architecture

Activity 1.1: Generate a Gemini API Key

- Visit Google AI Studio (<https://aistudio.google.com/app/apikey>) and sign in with your Google account.
- Click "Create API key", choose or create a Google Cloud project, and copy the generated key.
- Copy EduGenie/.env.example to EduGenie/.env and paste the key:
GEMINI_API_KEY=your_key_here
- The key is loaded automatically by gemini_client.py using python-dotenv. The .env file is listed in .gitignore so the key is never pushed to GitHub.

```
# Copy this file to ".env" (in this same EduGenie folder) and fill in your key.
# Get a free key at https://aistudio.google.com/app/apikey
GEMINI_API_KEY=your_gemini_api_key_here

# Gemini model to use. "gemini-flash-latest" always points to the newest Flash model.
# You can pin a specific model instead, e.g. GEMINI_MODEL=gemini-2.5-flash
GEMINI_MODEL=gemini-flash-latest

# Explanation backend: auto | local | gemini
# auto -> use local LaMini-Flan-T5 if requirements-local.txt is installed, else Gemini
EXPLAIN_BACKEND=auto

# Load the local model at startup instead of on the first "Explain" request (true/false)
PRELOAD_LOCAL_MODEL=false

LOG_LEVEL=INFO
```

Activity 1.2: Select the AI Models

Model	Used for	Why
Google Gemini (gemini-flash-latest, via google-genai SDK)	Q&A, summarization, quiz generation (JSON mode), learning paths, explanation fallback	Advanced reasoning, structured JSON output, fast cloud inference, free tier. The project guide's gemini-1.5-pro is retired, so the latest Flash model is used (configurable via GEMINI_MODEL).
MBZUAI/LaMini-Flan-T5-783M (local, optional)	Concept explanation	Instruction-tuned, lightweight, runs on CPU / Apple Silicon without API cost.

Activity 1.3: Define the Architecture (Folder Structure)

```
EduGenie-AI/
|-- EduGenie/
|   |-- main.py
|   |-- gemini_client.py
|   |-- explanation_module.py
|   |-- qna.py
|   |-- quiz_module.py
|   |-- summary_module.py
|   |-- learning_path.py
|   |-- templates/index.html
|   |-- static/style.css
|   |-- static/script.js
|   |-- tests/
|   |-- requirements.txt
|   |-- requirements-local.txt
|   |-- .env.example
|   # application
|   # FastAPI app + API routes
|   # shared Gemini client (google-genai)
|   # concept explanation (LaMini-Flan-T5 / Gemini)
|   # question answering
|   # quiz generation + JSON validation
|   # summarization
|   # learning recommendations
|   # frontend page
|   # styling
|   # frontend logic
|   # pytest tests + load_test.py
|   # core dependencies
|   # optional local model dependencies
|   # configuration template
```

```
| \-- pytest.ini  
|-- Project Documents/      # phase-wise project documents  
|-- .vscode/                # VS Code run / debug / test config  
|-- README.md               # setup, run and test guide
```

Activity 1.4: Set Up the Development Environment

```
python -m venv .venv  
.venv\Scripts\Activate.ps1      # Windows (macOS/Linux: source .venv/bin/activate)  
pip install -r EduGenie/requirements.txt  
# optional local explanation model (~3 GB):  
pip install -r EduGenie/requirements-local.txt
```

```
# Core EduGenie dependencies (Python 3.10+)  
fastapi>=0.115,<1.0  
uvicorn[standard]>=0.30  
jinja2>=3.1  
python-dotenv>=1.0  
google-genai>=1.0
```

```
# Testing  
pytest>=8.0  
httpx>=0.27
```

Milestone 2: Core Functionalities Development

Activity 2.1: Module Implementation

Gemini Client (gemini_client.py). Shared Gemini client: every feature calls one generate() function that loads the key, applies JSON mode / temperature and converts SDK errors into friendly messages.

```
def generate(prompt: str, *, json_mode: bool = False, temperature: float | None = None) -> str:
    """Send `prompt` to Gemini and return the response text.

    `json_mode=True` asks Gemini to reply with raw JSON (no Markdown).
    Raises GeminiNotConfiguredError / GeminiError on failure.
    """
    api_key = get_api_key()
    if api_key is None:
        raise GeminiNotConfiguredError(
            "Gemini API key is not configured. Copy EduGenie/.env.example to "
            "EduGenie/.env and set GEMINI_API_KEY, then restart the server."
        )

    from google.genai import types

    config = types.GenerateContentConfig(
        temperature=temperature,
        response_mime_type="application/json" if json_mode else None,
    )

    try:
        response = _client(api_key).models.generate_content(
            model=get_model_name(),
            contents=prompt,
            config=config,
        )
    except Exception as exc: # network errors, invalid key, quota, unknown model ...
        logger.error("Gemini request failed: %s", exc)
        # google-genai APIError exposes a short human-readable `.message`.
        message = getattr(exc, "message", None) or str(exc)
        raise GeminiError(f"Gemini request failed: {message}") from exc

    text = _extract_text(response)
    if not text:
        raise GeminiError(
            "Gemini returned an empty response (it may have been blocked by safety filters). "
            "Try rephrasing your input."
        )
    return text.strip()
```

Explanation Module (explanation_module.py). Uses LaMini-Flan-T5-783M when installed (EXPLAIN_BACKEND=auto/local); otherwise, or if the local model fails, it falls back to Gemini.

```
def explain_with_local_model(topic: str) -> str:
    import torch

    tokenizer, model = _load_local_model()
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = tokenizer(input_text, return_tensors="pt", truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model.generate(
            *inputs,
            max_new_tokens=200,
            temperature=0.7,
            top_k=50,
            top_p=0.95,
            do_sample=True,
        )
    return tokenizer.decode(outputs[0], skip_special_tokens=True).strip()

def explain_topic(topic: str) -> tuple[str, str]:
    """Return (explanation, source) where source is "local" or "gemini".

    Raises gemini_client.GeminiError if every available backend fails.
```

```
"""
topic = topic.strip()
backend = get_backend()

if backend == "local" or (backend == "auto" and local_model_available()):
    try:
        explanation = explain_with_local_model(topic)
        if explanation:
            return explanation, "local"
    except Exception:
        logger.exception("Local explanation model failed; falling back to Gemini")

return explain_with_gemini(topic), "gemini"
```

QnA Module (qna.py). Answers academic and general-knowledge questions concisely.

QNA_PROMPT = """You are EduGenie, a friendly and accurate AI tutor for students. Answer the student's question clearly and concisely (a short paragraph or a few bullet points). Use simple language, give a small example when it helps, and do not invent facts. If the question is not educational or is unclear, politely say so and suggest how to rephrase.

Question: {question}
 """

```
def answer_question_with_gemini(question: str) -> str:
    """Return a concise answer to `question`. Raises gemini_client.GeminiError on failure."""
    return gemini_client.generate(QNA_PROMPT.format(question=question.strip()), temperature=0.3)
```

Quiz Module (quiz_module.py). Generates MCQs with four options in JSON. clean_json_block() removes Markdown code fences and parse_quiz() validates every question (4 options, answer must match an option, letter answers mapped to text).

```
def clean_json_block(text: str) -> str:
    """Remove Markdown ``json code fences (if any) around the model output."""
    text = text.strip()
    match = re.search(r"```(?:json)?\s*\n?(.*?)```", text, flags=re.DOTALL)
    if match:
        text = match.group(1).strip()
    return text

def generate_quiz(text: str, num_questions: int = DEFAULT_NUM_QUESTIONS) -> list[dict]:
    """Generate a validated MCQ quiz from `text`. Raises gemini_client.GeminiError on failure."""
    n = max(1, min(int(num_questions), MAX_NUM_QUESTIONS))
    raw = gemini_client.generate(QUIZ_PROMPT.format(n=n, text=text.strip()), json_mode=True,
    temperature=0.4)
    return parse_quiz(raw)[:n]
```

Summary Module (summary_module.py). Summarizes long passages into an overview plus key points.

```
def summarize_text(text: str) -> str:
    """Return a concise summary of `text`. Raises gemini_client.GeminiError on failure."""
    return gemini_client.generate(SUMMARY_PROMPT.format(text=text.strip()), temperature=0.2)
```

Learning Path Module (learning_path.py). Creates a structured beginner-to-advanced plan adapted to the learner's level, with time estimates, resources, projects and a timeline.

```
def get_learning_recommendations(topic: str, level: str = "beginner") -> str:
    """Return a Markdown learning path for `topic`. Raises gemini_client.GeminiError on failure."""
    level = level.lower().strip() if level else "beginner"
    if level not in LEVELS:
        level = "beginner"
    prompt = LEARNING_PATH_PROMPT.format(topic=topic.strip(), level=level)
    return gemini_client.generate(prompt, temperature=0.5)
```

Activity 2.2: Backend API with FastAPI

Endpoint	Method	Input	Output
/qa	GET	question (query)	{question, answer}
/explain	POST	{topic}	{topic, explanation, source}
/quiz	POST	{text, num_questions}	{quiz: [{question, options, answer, explanation}]}
/summarize	POST	{text}	{summary}
/learn/recommendations	GET	topic, level (query)	{topic, level, recommendation}
/health	GET	-	{status, gemini_configured, gemini_model, ...}

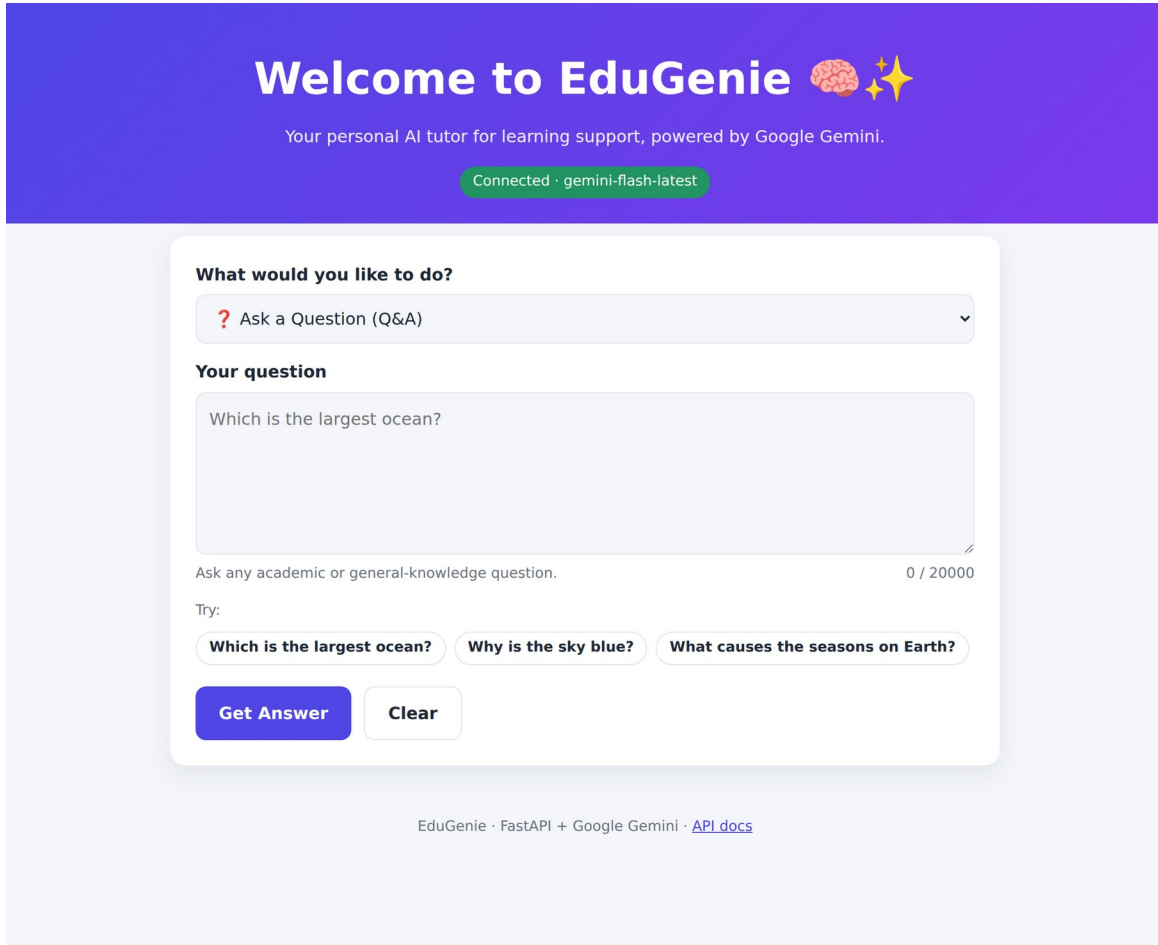
```
def validate_input(value: str | None, name: str, max_chars: int) -> str | JSONResponse:
    value = (value or "").strip()
    if not value:
        return error(f"Please provide {name}.", 400)
    if len(value) > max_chars:
        return error(f"Input is too long ({len(value)} characters). The limit is {max_chars}.", 400)
    return value

# Quiz Generation - POST API
@app.post("/quiz", tags=["learning"])
def quiz_api(body: QuizRequest):
    text = validate_input(body.text, "text or a topic for the quiz", MAX_TEXT_CHARS)
    if isinstance(text, JSONResponse):
        return text
    try:
        quiz = generate_quiz(text, body.num_questions)
        logger.info("Generated quiz with %d questions", len(quiz))
        return {"quiz": quiz}
    except gemini_client.GeminiError as exc:
        return ai_error(exc)
```

Milestone 3: Frontend Development

Activity 3.1: Build the Web Interface

- index.html (Jinja2): task dropdown (Q&A, Explain, Quiz, Summarize, Learning Path), text area with character counter, quiz size and learning-level options, example chips, Submit and Clear buttons, result card.
- style.css: responsive layout from mobile to desktop, gradient header, styled buttons/inputs/result container, automatic dark mode, quiz option states (correct = green, wrong = red).



The screenshot shows the EduGenie home page. At the top is a purple header with the text "Welcome to EduGenie" and a brain icon with two yellow stars. Below this is the subtitle "Your personal AI tutor for learning support, powered by Google Gemini." and a green status bar that says "Connected · gemini-flash-latest". The main content area is a white card with a light blue background. It features a dropdown menu labeled "What would you like to do?" with the option "Ask a Question (Q&A)". Below this is a text input field labeled "Your question" containing the text "Which is the largest ocean?". Under the input field is a character count "0 / 20000" and a prompt "Ask any academic or general-knowledge question." Below the input field are three example question chips: "Which is the largest ocean?", "Why is the sky blue?", and "What causes the seasons on Earth?". At the bottom of the card are two buttons: "Get Answer" (blue) and "Clear" (white).

Figure 2: EduGenie home page

Activity 3.2: Live Integration

- script.js sends a fetch() GET/POST request to the matching FastAPI endpoint when the form is submitted (or Ctrl/Cmd + Enter).
- A loading spinner is shown while the AI works; the result appears below the form in real time.
- Text results are rendered from Markdown with all HTML escaped (XSS-safe); quizzes are rendered as clickable options with feedback and score.
- Errors from the API (empty input, missing key, Gemini failure) are shown in a red message box.

Milestone 4: Deployment and Testing

Activity 4.1: Run Locally

```
cd EduGenie
uvicorn main:app --reload

# then open http://127.0.0.1:8000 (API docs: http://127.0.0.1:8000/docs)
```

In VS Code, the same can be done with Run and Debug -> "EduGenie: Run server (uvicorn)" -> F5.

Activity 4.2: Functional Testing

Automated tests (pytest) mock the Gemini API, so they run without a key or internet and verify every endpoint, validation rule and error path.

```
(.venv) EduGenie> pytest -v
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /home/user/EduGenie-AI/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/user/EduGenie-AI/EduGenie
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collecting ... collected 34 items

tests/test_api.py::test_index_page_renders PASSED [ 2%]
tests/test_api.py::test_static_files_served PASSED [ 5%]
tests/test_api.py::test_health PASSED [ 8%]
tests/test_api.py::test_qa PASSED [ 11%]
tests/test_api.py::test_qa_requires_question PASSED [ 14%]
tests/test_api.py::test_explain PASSED [ 17%]
tests/test_api.py::test_explain_trailing_slash_like_original_doc PASSED [ 20%]
tests/test_api.py::test_explain_empty_topic PASSED [ 23%]
tests/test_api.py::test_explain_topic_too_long PASSED [ 26%]
tests/test_api.py::test_summarize PASSED [ 29%]
tests/test_api.py::test_summarize_empty PASSED [ 32%]
tests/test_api.py::test_quiz PASSED [ 35%]
tests/test_api.py::test_quiz_num_questions PASSED [ 38%]
tests/test_api.py::test_quiz_invalid_num_questions PASSED [ 41%]
tests/test_api.py::test_quiz_invalid_ai_output PASSED [ 44%]
tests/test_api.py::test_quiz_empty PASSED [ 47%]
tests/test_api.py::test_learning_recommendations PASSED [ 50%]
tests/test_api.py::test_learning_recommendations_requires_topic PASSED [ 52%]
tests/test_api.py::test_gemini_failure_returns_502 PASSED [ 55%]
tests/test_api.py::test_missing_api_key_returns_503 PASSED [ 58%]
tests/test_live_gemini.py::test_live_qa SKIPPED (set RUN_LIVE_TESTS=...) [ 61%]
tests/test_live_gemini.py::test_live_quiz SKIPPED (set RUN_LIVE_TESTS=...) [ 64%]
tests/test_live_gemini.py::test_live_learning_path SKIPPED (set RUN_LIVE_TESTS=...) [ 67%]
tests/test_modules.py::test_clean_json_block_removes_fences PASSED [ 70%]
tests/test_modules.py::test_parse_quiz_accepts_wrapped_object_and_skips_bad_items PASSED [ 73%]
tests/test_modules.py::test_parse_quiz_extract_array_from_surrounding_text PASSED [ 76%]
tests/test_modules.py::test_parse_quiz_raises_on_garbage PASSED [ 79%]
tests/test_modules.py::test_quiz_format_error_is_gemini_error PASSED [ 82%]
tests/test_modules.py::test_learning_path_invalid_level_defaults_to_beginner PASSED [ 85%]
tests/test_modules.py::test_explain_falls_back_to_gemini_when_local_fails PASSED [ 88%]
tests/test_modules.py::test_explain_uses_local_model_when_selected PASSED [ 91%]
tests/test_modules.py::test_generate_without_key_raises PASSED [ 94%]
tests/test_modules.py::test_placeholder_key_is_treated_as_missing PASSED [ 97%]
tests/test_modules.py::test_model_name_default_and_override PASSED [100%]

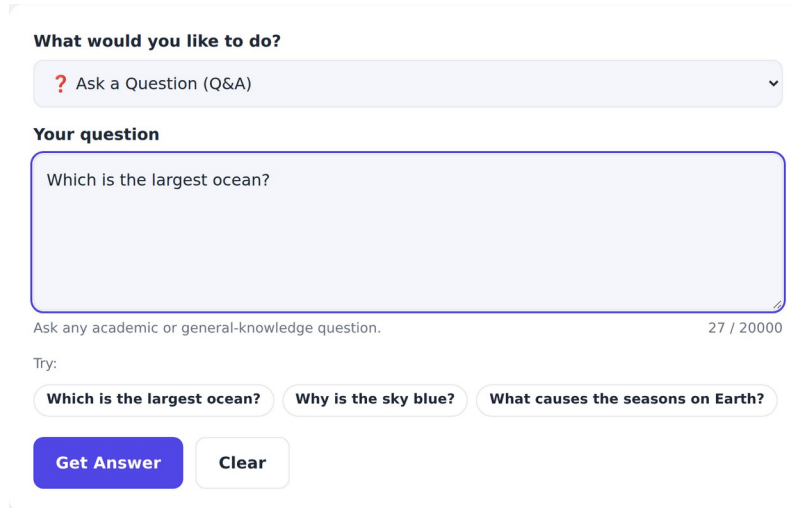
===== 31 passed, 3 skipped in 0.18s =====
```

Figure 3: pytest results - 31 passed (3 live tests run only with RUN_LIVE_TESTS=1)

Exploring EduGenie:

Asking questions:

Select "Ask a Question", type a question such as "Which is the largest ocean?" and click Get Answer.



The interface shows a dropdown menu under "What would you like to do?" with the option "Ask a Question (Q&A)" selected. Below this is a text input field labeled "Your question" containing the text "Which is the largest ocean?". At the bottom, there are buttons for "Get Answer" and "Clear".

What would you like to do?

? Ask a Question (Q&A)

Your question

Which is the largest ocean?

Ask any academic or general-knowledge question. 27 / 20000

Try:

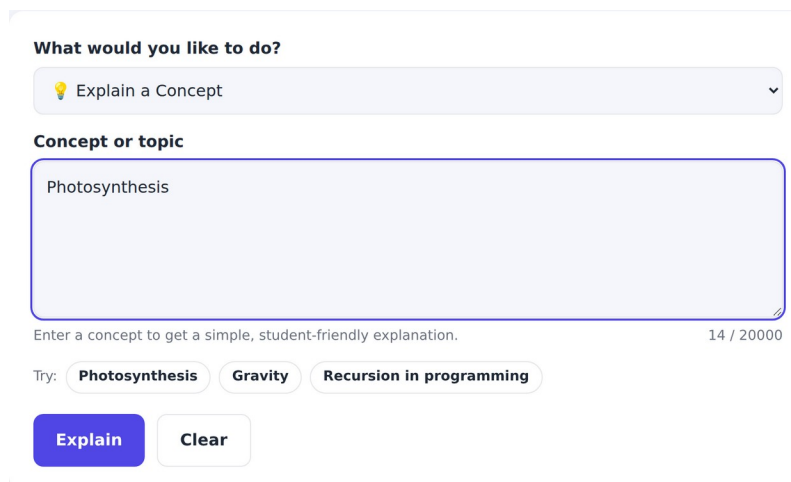
Which is the largest ocean? Why is the sky blue? What causes the seasons on Earth?

Get Answer Clear

[Insert screenshot of the live AI result for "Asking questions" after running EduGenie with your Gemini API key]

Explanation of any topic:

Select "Explain a Concept" and enter a topic such as Photosynthesis. The badge shows whether Gemini or the local LaMini-Flan-T5 model answered.



The interface shows a dropdown menu under "What would you like to do?" with the option "Explain a Concept" selected. Below this is a text input field labeled "Concept or topic" containing the text "Photosynthesis". At the bottom, there are buttons for "Explain" and "Clear".

What would you like to do?

💡 Explain a Concept

Concept or topic

Photosynthesis

Enter a concept to get a simple, student-friendly explanation. 14 / 20000

Try: Photosynthesis Gravity Recursion in programming

Explain Clear

[Insert screenshot of the live AI result for "Explanation of any topic" after running EduGenie with your Gemini API key]

Summarising long paragraphs:

Paste a long passage and click Summarize to get a one-line overview and key bullet points.

What would you like to do?

Summarize a Passage

Text to summarize

The water cycle describes how water evaporates from the surface of the earth, rises into the atmosphere, cools and condenses into clouds, and falls again to the surface as precipitation. Water falling on land collects in rivers and lakes, soil, and porous layers of rock, and much of it flows back into the oceans, where it will once more evaporate. The cycling of water in and out of the atmosphere is a significant aspect of the weather patterns on Earth.

Paste long educational content to get a concise revision summary. 457 / 20000

Try: The water cycle describes how water e...

Summarize Clear

[Insert screenshot of the live AI result for "Summarising long paragraphs" after running EduGenie with your Gemini API key]

Generating quizzes:

Enter a topic or passage, choose 3, 5 or 10 questions and click Generate Quiz. Each question has 4 options; clicking an option shows whether it is correct (green) or wrong (red), the explanation and the running score.

What would you like to do?

Generate a Quiz

Topic or passage

The Pythagoras Theorem

Enter a topic or paste a passage to generate multiple-choice questions. 22 / 20000

Number of questions

3

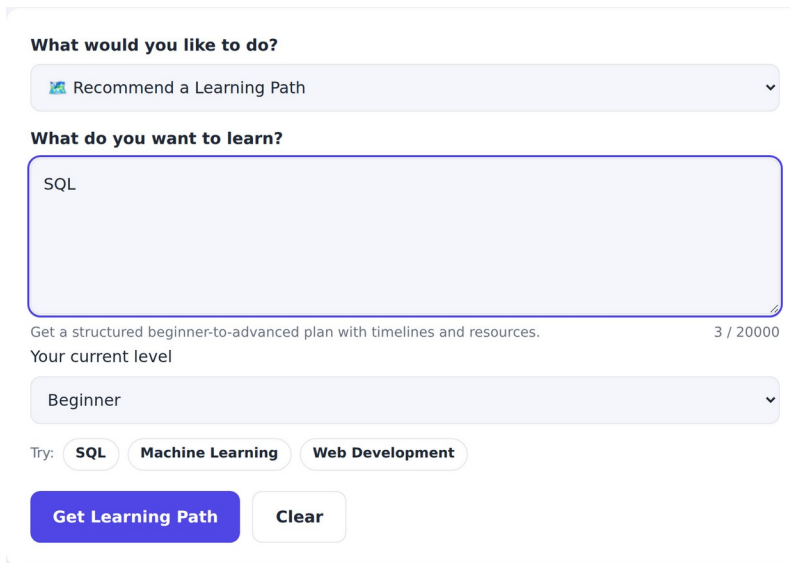
Try: The Pythagoras Theorem Solar System World War II

Generate Quiz Clear

[Insert screenshot of the live AI result for "Generating quizzes" after running EduGenie with your Gemini API key]

Learning recommendations from beginner to advanced:

Enter a topic (e.g. SQL), choose the current level and click Get Learning Path to get topics, timelines, resources, practice projects and a weekly plan.

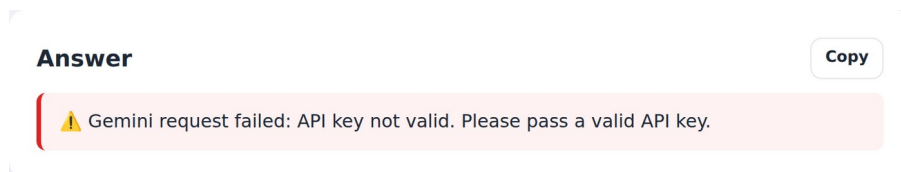


The screenshot shows a web interface for generating learning paths. It has two main sections: 'What would you like to do?' with a dropdown menu set to 'Recommend a Learning Path', and 'What do you want to learn?' with a text input field containing 'SQL'. Below the input field, there is a small text description: 'Get a structured beginner-to-advanced plan with timelines and resources.' and a character count '3 / 20000'. Underneath, there is a section for 'Your current level' with a dropdown menu set to 'Beginner'. At the bottom, there are three radio buttons labeled 'SQL', 'Machine Learning', and 'Web Development', with 'SQL' selected. Finally, there are two buttons: 'Get Learning Path' (in blue) and 'Clear' (in white).

[Insert screenshot of the live AI result for "Learning recommendations from beginner to advanced" after running EduGenie with your Gemini API key]

Error handling:

If the API key is missing or invalid, or Gemini is unavailable, EduGenie shows a clear message instead of crashing:



The screenshot shows an error message box. At the top left, it says 'Answer'. At the top right, there is a 'Copy' button. The main body of the message is a red box with a yellow warning triangle icon and the text: 'Gemini request failed: API key not valid. Please pass a valid API key.'

Mobile view:

Welcome to EduGenie



Your personal AI tutor for learning support,
powered by Google Gemini.

Connected · gemini-flash-latest

What would you like to do?

? Ask a Question (Q&A) ▾

Your question

Which is the largest ocean?

Ask any academic or general-
knowledge question. 0 / 20000

Try:

Which is the largest ocean?

Why is the sky blue?

What causes the seasons on Earth?

Get Answer

Conclusion:

EduGenie is a robust and accessible AI-powered educational assistant that combines Google Gemini's cloud intelligence with an intuitive, low-footprint web interface. It simplifies complex concepts, answers questions instantly, generates interactive quizzes, summarizes long passages and builds personalized learning paths - making quality learning support available to anyone with a browser.

The project achieved its milestones in AI integration, prompt engineering, API design, user-experience design and testing. Challenges such as keeping AI output reliable (solved with JSON mode and strict quiz validation), handling API errors gracefully, keeping the API key secure and supporting low-resource devices (optional local model, stateless design) provided valuable learning outcomes.

Future enhancements include voice-based interaction, multilingual support, a mobile application, progress-tracking dashboards, gamification (badges, streaks), user accounts with history, LMS integration (Moodle, Google Classroom) and image/PDF input for snapshot-based doubt solving.

Submitted by: Arumparithi B